

# Aktiviteti 5&6 — Funksionet (Vazhdim)

## Udhëzues

---

Herën e kaluar mësuam si të deklarojmë dhe të thërrasim një funksion, dhe dallimin mes variablave lokale dhe globale. Sot shkojmë një hap më tutje: si t'i japim një funksioni të dhëna nga jashtë (**parametra**), si të marrim një vlerë prapa nga ai (**return**), dhe një mënyrë tjetër, më e shkurtër, për t'i shkruar funksionet — **Arrow Functions**.

## 1. Parametrat e Funksionit

---

Deri tani funksionet tona bënë gjithmonë të njëjtën gjë, pa marrë parasysh kush i thërret. Por shpesh duam që një funksion të sillet ndryshe në bazë të të dhënave që i japim — këtu na duhen **parametrat**.

```
function pershendetPerdoruesin(emri) {  
  console.log("Përshëndetje, " + emri + "!");  
}  
  
pershendetPerdoruesin("Arta");  
pershendetPerdoruesin("Blerim");
```

`emri` këtu është parametri — një "vend-mbajtës" brenda kllapave `()` të funksionit. Sa herë që e thërrasim funksionin, i japim një vlerë të ndryshme (të quajtur **argument**), dhe funksioni e përdor atë vlerë brenda trupit të tij.



Një funksion mund të ketë sa parametra të dojë, të ndarë me presje: `function mbledh(a, b, c) { ... }`.

## 2. Return

---

Deri tani, funksionet tona vetëm printonin diçka në console. Por shumë herë duam që funksioni të na **kthejë** një vlerë, në mënyrë që ta ruajmë në një variabël dhe ta përdorim më vonë. Kjo bëhet me `return`:

```
function mblidhDyNumra(a, b) {  
  return a + b;  
}  
  
let rezultati = mblidhDyNumra(5, 3);  
console.log(rezultati); // 8
```

⚠ Sapo funksioni arrin të `return`, ai ndalon ekzekutimin dhe del jashtë — çdo kod pas `return` brenda funksionit **nuk ekzekutohet** fare.

### 3. Arrow Functions

**Arrow Functions** janë një mënyrë tjetër, më e shkurtër, për të shkruar funksione — përdorin `=>` në vend të fjalës `function`. Bëjnë saktësisht të njëjtën punë, thjesht sintaksa ndryshon pak varësisht sa parametra ka funksioni dhe sa rreshta ka brenda.

#### Tipi 1 — një rresht, një parametër (return i nënkuptuar)

Kur funksioni ka **vetëm një parametër** dhe trupi është **vetëm një rresht**, mund t'i heqim edhe kllapat `()` rreth parametrit, edhe `{ }`, edhe vetë fjalën `return` — vlera e rreshtit kthehet automatikisht:

```
const dyfishoje = numri => numri * 2;  
  
console.log(dyfishoje(5)); // 10
```

#### Tipi 2 — më shumë rreshta, një parametër

Kur trupi ka më shumë se një rresht, na duhen `{ }` përreth tij, dhe `return` duhet ta shkruajmë eksplicit (nuk kthehet më automatikisht). Parametri i vetëm mund të mbetet pa kllapa, por shumica e kodit zakonisht i vendos kllapat gjithsesi për qartësi:

```
const pershendetPerdoruesin = (emri) => {  
  const mesazhi = "Përshëndetje, " + emri + "!";  
  console.log(mesazhi);  
  return mesazhi;  
};  
  
pershendetPerdoruesin("Dea");
```

#### Tipi 3 — më shumë rreshta, më shumë se një parametër

Kur funksioni ka **dy ose më shumë parametra**, kllapat `()` rreth tyre janë të **detyrueshme**:

```
const llogaritCmimin = (cmimiBaze, sasia) => {  
  const totali = cmimiBaze * sasia;  
  return totali;  
};  
  
console.log(llogaritCmimin(2.5, 4)); // 10
```

💡 Rregull i thjeshtë për t'u mbajtur mend: **1 parametër + 1 rresht** → mund t'i heqësh të gjitha kllapat e tepërta. Sapo shton qoftë edhe një rresht më shumë, qoftë edhe një parametër më shumë — kthehen kllapat.

## Përmbledhje e shpejtë

```
function emri(parametra) { return vlera; } // funksion klasik  
  
const emri = param => vlera; // Arrow – 1 rresht, 1 param  
const emri = (param) => { return vlera; }; // Arrow – shumë rreshta, 1 param  
const emri = (p1, p2) => { return vlera; }; // Arrow – shumë rreshta, 2+ param
```

## Sfidat e tua

### Sfidë e lehtë

Rishkruani funksionin `mblidhDyNumra` nga seksioni 2 si Arrow Function, në format një-rresht (Tipi 1 — por kujdes, ky rast ka 2 parametra, jo 1 — çfarë ndryshon në sintaksë krahasuar me Tipin 1 të mësipërm?).

### Sfidë më e vështirë

Shkruani një Arrow Function `eshteMoshRritur` me një parametër (`mosha`), që kthen `true` nëse mosha është 18 ose më shumë, përndryshe `false`. Shkruajeni fillimisht si Tipi 2 (me `{ }` dhe `return` eksplisit), pastaj provoni ta shkurtoni edhe në Tipin 1 duke përdorur operatorin ternar brenda return-it të nënkuptuar (p.sh. `const emri = param => kushti ? "a" : "b";`).